

App Dev Project Report

Placement Portal — Web Application

1. Student Details

Name: Sinan Ameer S H

Roll Number: 21f3000581

Email: 21f3000581@ds.study.iitm.ac.in

About Me: I am a Diploma level student at IIT Madras BS Degree program with a deep interest in data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Placement Portal

Problem Statement

To design and build a multi-role web application that enables students, companies, and an admin to manage the complete placement lifecycle — including student registration with admin approval, company onboarding, placement drive creation, application tracking, and status management including blacklisting.

Approach

The app was built using Flask as the backend framework with a modular, Blueprint-based structure. It supports three distinct user roles (Admin, Company, Student), each requiring admin approval before access. SQLAlchemy manages all database operations with SQLite. Role-based access is enforced using custom decorators, and WTForms with Flask-WTF provides secure, validated forms with CSRF protection throughout the application.

3. AI/LLM Declaration

I used ChatGPT (GPT-4) to assist in writing SQLAlchemy model definitions, creating WTForms classes, and improving variable naming consistency.

The extent of AI/LLM usage is around 15-20%, limited to code suggestions and template formatting. All final implementation logic, debugging and Blueprint wiring were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework with Blueprint support
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Flask-Login	User authentication and session management
WTForms / Flask-WTF	Frontend form validation with CSRF protection
Werkzeug	Secure password hashing utilities
SQLite	Lightweight local database for storing all portal data

5. Database Schema / ER Diagram

Tables

1. User — login credentials and role (id, username, email, password_hash, role, is_active)
2. CompanyProfile — company details (id, user_id, company_name, website, location, description, hr_contact, approval_status)
3. StudentProfile — student academic info (id, user_id, full_name, cgpa, resume_link, skills, contact_number, place_of_residence, approval_status)
4. PlacementDrive — job postings (id, company_id, job_title, job_description, criteria, salary, location, status, date_posted, deadline)
5. Application — student applications (id, student_id, drive_id, date_applied, status)

Relationships

- One-to-One: User -> CompanyProfile (via user_id FK)
- One-to-One: User -> StudentProfile (via user_id FK)
- One-to-Many: CompanyProfile -> PlacementDrive
- One-to-Many: StudentProfile -> Application
- One-to-Many: PlacementDrive -> Application

Approval Status Values

Model	approval status values	Meaning
CompanyProfile	pending / approved / rejected / blacklisted	Set by admin on company registration and via blacklist action
StudentProfile	pending / approved / rejected / blacklisted	Set by admin on student registration and via blacklist action

PlacementDrive	pending / approved / rejected	Set by admin before drive becomes visible to students
Application	applied / shortlisted / selected / rejected	Set by company when reviewing student applications



6. API Resource Endpoints

Auth Routes

Endpoint	Method	Description
/login	GET/POST	Render and process user login form
/register	GET/POST	Register as Student or Company; both require admin approval
/logout	GET	Log out the current user

Student Routes (/student/...)

Endpoint	Method	Description
/student/dashboard	GET	View approved drives with open deadlines and own application statuses
/student/apply/<drive_id>	GET	Apply to an approved drive (requires complete profile with resume link)

/student/profile	GET/POST	View and update CGPA, skills, contact number, residence, and resume link
------------------	----------	--

Company Routes (/company/...)

Endpoint	Method	Description
/company/dashboard	GET	View all placement drives posted by the company
/company/create_drive	GET/POST	Create a new placement drive (defaults to pending admin approval)
/company/drive/<id>/applications	GET	View all student applications for a specific drive
/company/application/<id>/update/<status>	GET	Update application status: shortlisted / selected / rejected

Admin Routes (/admin/...)

Endpoint	Method	Description
/admin/dashboard	GET	Overview: pending companies, pending students, pending drives, stats
/admin/approve_company/<id>	GET	Approve company: sets is_active=True, approval_status='approved'
/admin/reject_company/<id>	GET	Reject company: sets approval_status='rejected' (record kept)
/admin/approve_student/<id>	GET	Approve student: sets is_active=True, approval_status='approved'
/admin/reject_student/<id>	GET	Reject student: sets approval_status='rejected'
/admin/approve_drive/<id>	GET	Approve placement drive for student visibility
/admin/reject_drive/<id>	GET	Reject a placement drive
/admin/students	GET	List all registered student profiles
/admin/companies	GET	List all registered company profiles
/admin/blacklist_student/<id>	GET	Blacklist student: sets is_active=False, approval_status='blacklisted'
/admin/blacklist_company/<id>	GET	Blacklist company: sets is_active=False, approval_status='blacklisted'

7. Architecture and Features

Architecture Overview

- run.py — application entry point, calls create_app()
- config.py — configuration: secret key, DB URI, SQLAlchemy settings
- app/__init__.py — app factory: initializes db, login_manager, registers all 5 Blueprints
- app/models.py — SQLAlchemy models: User, CompanyProfile, StudentProfile, PlacementDrive, Application
- app/forms.py — WTForms: LoginForm, RegistrationForm (with hr_contact, contact_number, place_of_residence), DriveForm, StudentProfileForm
- app/decorators.py — role guards: admin_required, company_required, student_required
- app/controllers/ — Blueprints: auth, admin, company, student, main
- app/templates/ — Jinja2 templates organized by role: auth/, admin/, company/, student/
- app/static/ — CSS stylesheets and static assets

Implemented Features

- Multi-role registration: Students and Companies both register and wait for admin approval before login access
- Role-based login with Flask-Login; redirects to role-specific dashboard after authentication
- Admin Dashboard: live stats (total students, companies, drives) and actionable pending queues for companies, students, and drives
- Admin can approve or reject both company and student registrations, with approval_status tracked on each profile
- Admin can blacklist approved students or companies, revoking access (is_active=False) at any time
- Admin can list all student profiles (/admin/students) and all company profiles (/admin/companies)
- Company Dashboard: view all created drives with their current approval status
- Company can create placement drives (submitted as pending, visible to students only after admin approval)
- Company can view applicants per drive and update each application status: shortlisted / selected / rejected
- Student Dashboard: browse active approved drives with open deadlines; view own application statuses
- Students must complete their profile (resume link required) before applying to any drive
- Duplicate application prevention: students cannot apply more than once to the same drive
- Extended profiles: CompanyProfile captures hr_contact; StudentProfile captures contact_number and place_of_residence.

Video Link:

<https://drive.google.com/file/d/1P2n23p6vIxqHwLLJlIryEXDckGKm03SB/view?usp=sharing>

Default admin credentials:

- Email: admin@institute.edu
- Password: admin123